Introduction to Spring, Spring Boot, and Maven

The original materials have been reorganized into a more technically and pedagogically appropriate sequence. In particular, the **Spring Framework is introduced before Spring Boot**, and the **Maven fundamentals are covered before using Spring Initializr**, since Spring Initializr generates a Maven or Gradle project and you should understand the resulting structure.

Learning Objectives

By the end of these notes, students should be able to:

- Explain the purpose and major goals of the Spring Framework.
- Describe the relationship between Spring Framework and Spring Boot.
- Distinguish between Spring Framework modules and Spring projects.
- Explain the role of JavaBeans and Spring-managed beans.
- Describe how Maven manages project dependencies and builds.
- Identify the standard structure of a Maven project.
- Interpret important elements of a pom.xml file.
- Explain Maven coordinates: groupId, artifactId, and version.
- Generate a Spring Boot project using Spring Initializr.
- Run a Spring Boot application with an embedded web server.
- Create and test a basic REST controller.

---

1. Introduction to the Spring Framework

1.1 What Is Spring?

The **Spring Framework** is a Java application framework designed to simplify the development of enterprise and web applications.

Spring provides infrastructure for common development concerns such as:

- object creation and management;
- dependency injection;
- web application development;
- database access;
- transaction management;
- testing;
- aspect-oriented programming.

One of Spring's central ideas is that application classes should remain as simple and independent as possible.

POJOs

Spring encourages development using **POJOs**, or **Plain Old Java Objects**.

A POJO is essentially an ordinary Java class that does not need to inherit from a special framework class simply to participate in the application architecture.

For example:

```java
public class CustomerService {

    public void processCustomer() {
        System.out.println("Processing customer...");
    }
}
```

There is nothing Spring-specific about this class. Spring can nevertheless create, configure, and manage an instance of it.

Note — POJO vs. Spring Bean

A POJO describes the nature of a Java class. A **Spring bean** is an object whose lifecycle is managed by the Spring container. A POJO can therefore become a Spring bean when Spring manages its instance.

---

2. Goals of the Spring Framework

Spring was designed to make Java enterprise development simpler and more maintainable.

Three important goals are particularly relevant.

2.1 Lightweight Development with POJOs

Spring allows developers to build applications using ordinary Java objects rather than requiring heavyweight enterprise components.

This makes application code:

- easier to write;
- easier to understand;
- easier to test;
- less dependent on framework-specific APIs.

2.2 Loose Coupling Through Dependency Injection

Consider the following class:

```
public class OrderService {

    private PaymentService paymentService =
            new PaymentService();
}
```

`OrderService` creates its own **`PaymentService`**.

This produces relatively **tight coupling**, because **`OrderService`** directly controls the creation of its dependency.

Spring promotes a different approach:

```
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

The dependency is supplied from outside the class.

This concept is known as **Dependency Injection (DI)**.

Spring's container can create both objects and inject the appropriate dependency.

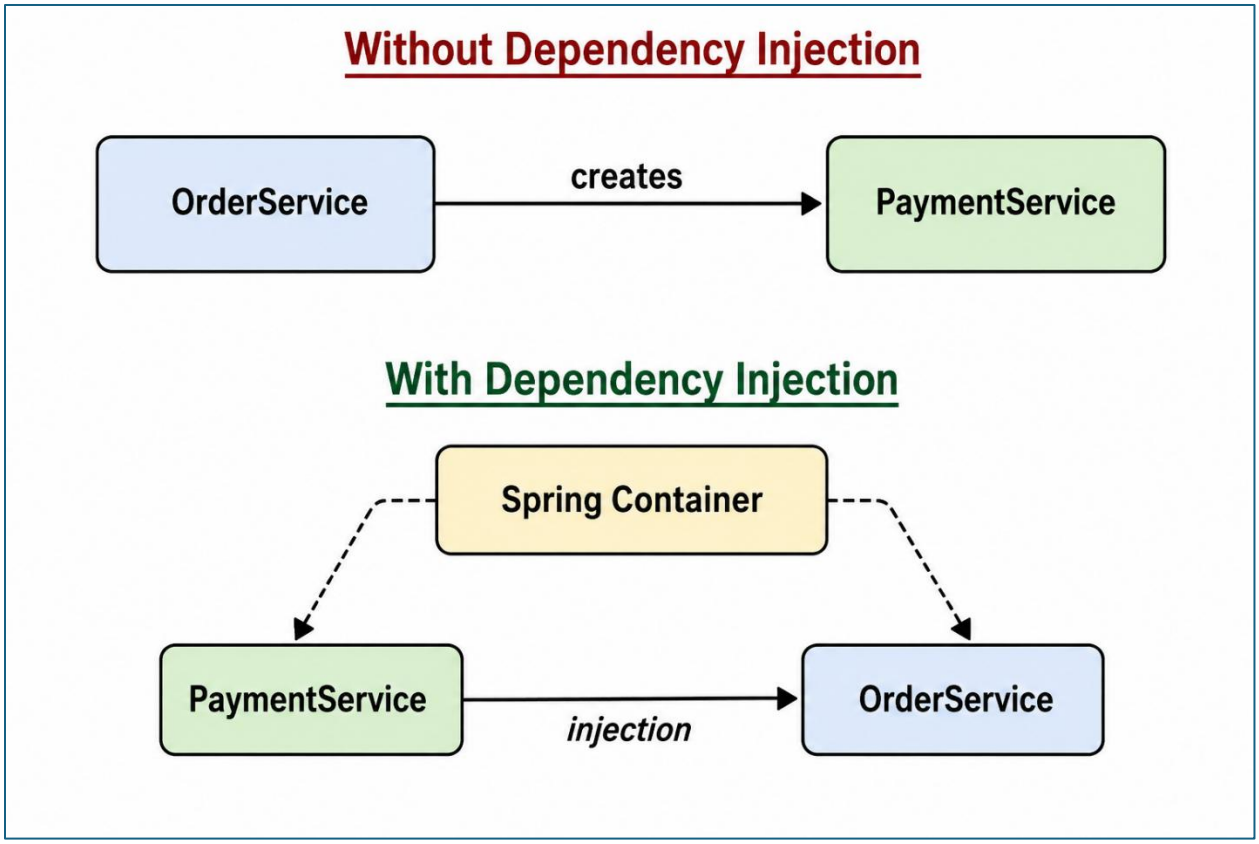
Conceptually:

```
Spring Container
        |
        +---- creates PaymentService
        |
        +---- creates OrderService
                        |
                        +---- injects PaymentService
```

Tip

A useful way to understand Dependency Injection is:

The object uses its dependencies but does not need to be responsible for creating them.



2.3 Reducing Boilerplate Code

Enterprise applications frequently require repetitive infrastructure code for tasks such as:

- database connections;
- transactions;
- configuration;
- object creation;
- testing.

Spring provides abstractions and helper classes that reduce much of this repetitive code.

---

3. Major Areas of the Spring Framework

The Spring Framework consists of several major functional areas.

3.1 Core Container

The **Core Container** provides the foundation of Spring.

It is responsible for managing application objects—commonly called **beans**—and their dependencies.

Important concepts include:

- bean management;
- application context;
- configuration;
- dependency injection;
- Spring Expression Language (SpEL).

The basic idea is:

```
Configuration
      ↓
Spring Container
      ↓
Creates and manages beans
      ↓
Injects dependencies
```

3.2 Aspect-Oriented Programming

Aspect-Oriented Programming (AOP) is useful for functionality that applies across multiple areas of an application.

Examples include:

- logging;
- security;
- transaction management;
- instrumentation.

Instead of duplicating these concerns throughout application classes, AOP allows them to be applied more systematically.

3.3 Data Access and Integration

Spring provides support for communicating with databases and messaging systems.

Important technologies include:

JDBC: Spring provides helper functionality that simplifies Java database access.

ORM: Spring integrates with Object-Relational Mapping technologies such as Hibernate and JPA.

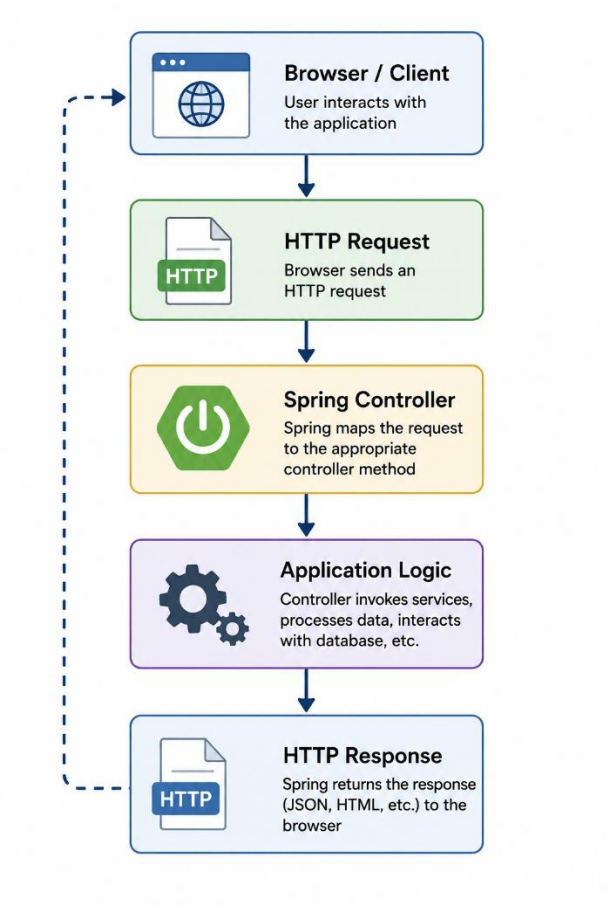
JMS: Java Message Service support enables asynchronous messaging.

Transaction Management: Spring provides infrastructure for managing database transactions.

3.4 Web Development

Spring supports Java web applications through its web technologies, including Spring MVC and REST controllers.

A typical web request may conceptually follow this path:



We will implement a simple controller later in this lesson.

3.5 Instrumentation

Spring also provides support for advanced capabilities involving:

- class loaders;
- Java agents;
- JMX monitoring.

These capabilities are generally relevant to more advanced enterprise applications.

3.6 Testing

Spring provides support for:

- unit testing;
- mock objects;
- integration testing;
- testing Spring-managed components.

This makes it easier to verify application behavior without deploying the complete production system.

---

4. Spring Projects

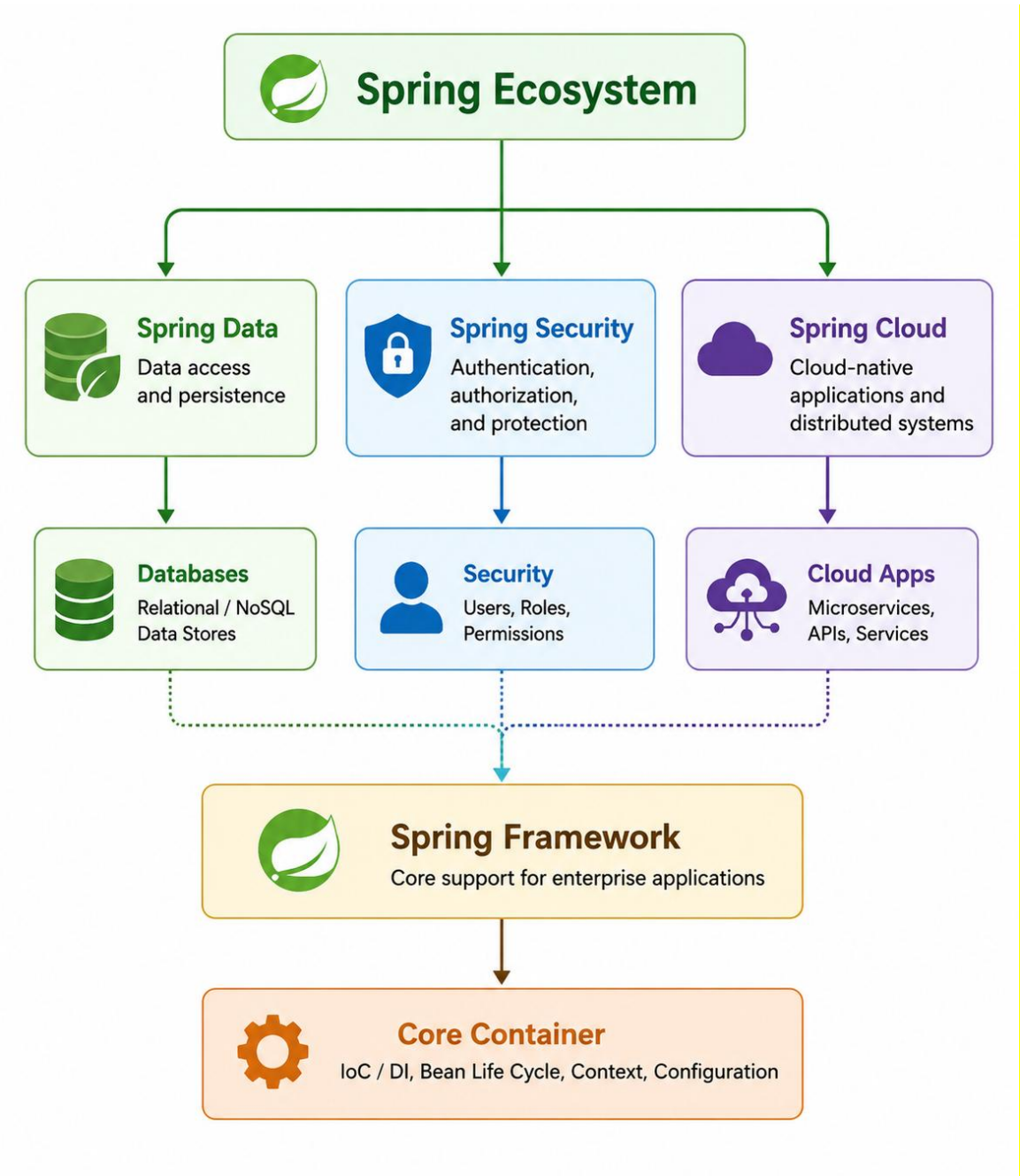
The **Spring Framework** is only one part of the larger Spring ecosystem.

Spring also provides specialized projects that build on or integrate with the core framework.

Examples include:

Project	Main Purpose
Spring Cloud	Cloud-native and distributed applications
Spring Data	Data-access technologies
Spring Batch	Batch-processing applications
Spring Security	Authentication and authorization
Spring Web Services	Web-service development
Spring LDAP	LDAP integration

These projects are optional. Developers select the technologies appropriate for a particular application.



Note

Do not confuse **Spring Framework modules** with **Spring projects**. Framework modules are parts of the Spring Framework itself, whereas projects such as Spring Data and Spring Security provide specialized functionality within the broader Spring ecosystem.

5. JavaBeans and Spring Beans

Before examining how Spring manages objects, it is useful to understand the JavaBean convention.

5.1 What Is a JavaBean?

A **JavaBean** is a Java class designed according to a set of established conventions.

A typical JavaBean contains:

- a public no-argument constructor;
- private properties;
- public getter and setter methods;
- standard property naming conventions.

Historically, JavaBeans may also implement Serializable, particularly when their state needs to be persisted or transferred.

For example:

```java
public class Student {

    private String name;
    private int age;

    public Student() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

The fields are private:

`private String name;`

`private int age;`

but accessible through public methods:

`getName()`

`setName(...)`

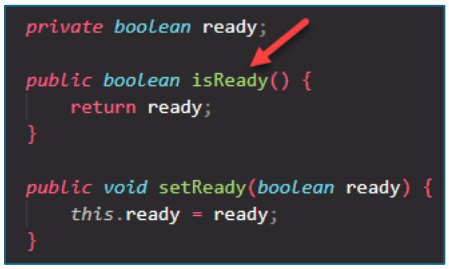
`getAge()`

`setAge(...)`

This preserves encapsulation while allowing frameworks and development tools to work with the properties.

Boolean Properties

For a Boolean property, a getter may follow the `isXxx()` convention:



Technical Note

Serializable should not be treated as an absolute requirement for every modern JavaBean-style class. It is primarily necessary when object serialization is actually required.

---

6. How Spring Manages Beans

The term **bean** has a particularly important meaning in Spring.

A **Spring bean** is an object created and managed by the Spring container.

Spring Boot simplifies the configuration and discovery of these objects.

6.1 Component Scanning

Spring can automatically discover classes marked with annotations such as:

`@Component`

`@Service`

`@Repository`

`@Controller`

`@RestController`

These classes can then be registered with the Spring container.

6.2 Java-Based Bean Configuration

Beans can also be defined explicitly using Java configuration.

For example:

```java
@Configuration
public class AppConfig {

    @Bean
    public MyService myService() {
        return new MyService();
    }
}
```

Here **`@Configuration`** identifies a configuration class.

The method **`@Bean`** tells Spring that the returned object should be managed as a bean.

6.3 Dependency Injection

Once objects are managed by Spring, they can be injected into other components.

Spring supports:

- constructor injection;
- setter injection;
- field injection.

Constructor injection conceptually looks like:

```java
public MyController(MyService myService) {
    this.myService = myService;
}
```

Spring supplies the required **`MyService`** object.

6.4 Externalized Configuration

Spring Boot allows configuration values to be placed outside Java source code in files such as **application.properties** or application.yml.

Configuration can then be bound to Java objects using facilities such as **@ConfigurationProperties**

6.5 Profiles

Different application environments may require different configurations.

Examples include:

- development
- testing
- production

Spring supports environment-specific configuration and beans through **profiles**, including the **@Profile** annotation.

6.6 Bean Scopes

Spring supports different bean lifecycles or **scopes**.

Examples include:

- *singleton* — normally one bean instance per Spring application context;
- *prototype* — creates a new instance when requested;
- *request* — associated with an HTTP request;
- *session* — associated with an HTTP session.

Pitfall

Do not assume that **JavaBean** and **Spring bean** mean exactly the same thing. A JavaBean describes a Java class convention, while a Spring bean refers specifically to an object managed by the Spring container.

---

7. Introduction to Spring Boot

7.1 The Problem Spring Boot Addresses

The Spring Framework is powerful, but traditionally starting a Spring application required developers to make several setup decisions.

For example:

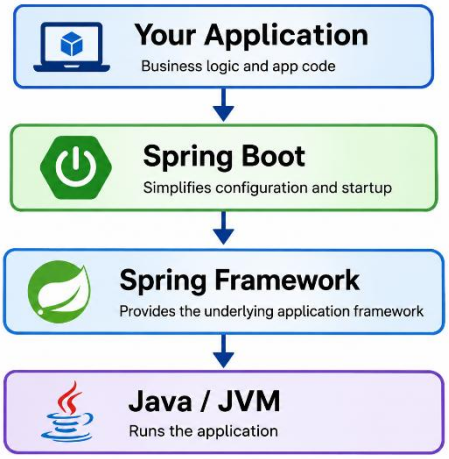
- Which dependencies are required?
- Which JAR files should be included?
- How should Spring be configured?
- Should configuration use XML or Java?
- Which application server should be installed?
- How should the application be deployed?

Spring Boot was created to simplify much of this setup.

---

8. What Is Spring Boot?

Spring Boot builds on top of the Spring Framework and simplifies the process of creating Spring applications.



It does not replace Spring.

Instead, it provides mechanisms such as:

- auto-configuration;
- starter dependencies;
- convention over configuration;
- embedded web servers;
- externalized configuration;
- simplified application startup.

---

9. Spring Boot Auto-Configuration

One of Spring Boot's major features is **auto-configuration**.

Spring Boot examines factors such as:

- dependencies available on the classpath;
- application configuration;
- existing beans;

and attempts to configure appropriate Spring components automatically.

For example, adding web-related Spring Boot dependencies can cause Spring Boot to configure much of the infrastructure required for a web application.

Note

Auto-configuration does not mean that Spring Boot magically knows the application's business requirements. It provides sensible default infrastructure configuration that developers can customize when necessary.

---

10. Embedded Web Servers

Traditional Java web applications often required developers to separately install and configure an application server.

Spring Boot can instead package a web server with the application.

Depending on the Spring Boot setup, supported embedded servers can include:

- Tomcat;
- Jetty;
- Undertow.

Conceptually, instead of:

```
External Server
    |
    +-- Application
```

Spring Boot allows:

```
Application JAR
    |
    +-- Application Code
    |
    +-- Embedded Server
```

The application can therefore be started as a standalone Java application.

For example:

```
java -jar mycoolapp.jar
```

Tip

The embedded-server model is one of the reasons Spring Boot applications are convenient to develop, test, package, and deploy.

---

11. Relationship Between Spring and Spring Boot

A common misunderstanding is that Spring Boot replaces Spring technologies.

It does not.

Spring Boot uses Spring technologies underneath.

For example, Spring Boot applications may use:

- Spring Core;
- Spring MVC;
- Spring REST capabilities;
- Spring Data;
- Spring Security.

Spring Boot primarily simplifies **configuration, startup, packaging, and integration**.

Frequently Asked Questions

1. Does Spring Boot replace Spring MVC or REST technologies?

No. Spring Boot can configure and use those technologies.

2. Is Spring Boot inherently faster at runtime than Spring?

Not simply because it is Spring Boot. It ultimately uses Spring Framework components underneath.

3. Do I need a special IDE?

No.

Spring Boot projects can be developed using:

- IntelliJ IDEA;
- Eclipse;
- NetBeans;
- VS Code or other editors;
- command-line development tools.

---

12. Introduction to Maven

Before generating our first Spring Boot project, we need to understand Maven because Spring Initializr can generate a Maven-based project.

12.1 What Is Maven?

Apache Maven is a build and dependency-management tool commonly used for Java projects.

Among other tasks, Maven can:

- manage dependencies;
- compile source code;
- execute tests;
- package applications;
- execute build plugins;
- provide a standardized project structure.

---

13. The Dependency Management Problem

Suppose an application requires several external libraries.

Without a dependency-management system, developers might have to:

- locate each library;
- download its JAR files;
- determine the correct versions;
- manually add them to the classpath;
- identify additional libraries required by those libraries.

This quickly becomes difficult.

Maven automates much of this process.

---

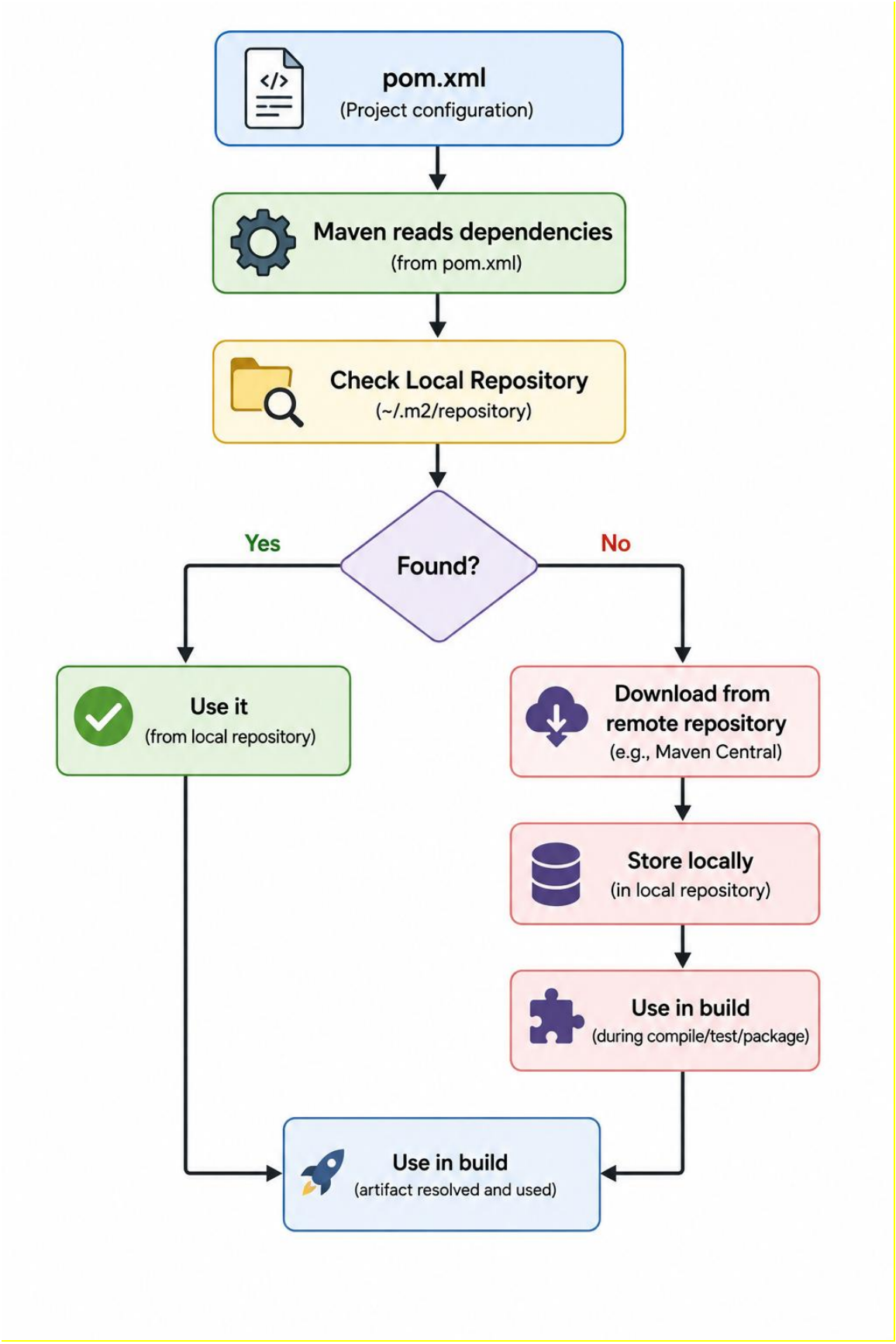
14. How Maven Resolves Dependencies

Developers declare dependencies in Maven's configuration file:

`pom.xml`

Maven then resolves the required artifacts.

The process can be represented as:


pom.xml
(Project configuration)
Maven reads dependencies
(from pom.xml)
Check Local Repository
(~/.m2/repository)
Found?
Yes
No
Use it
(from local repository)
Download from
remote repository
(e.g., Maven Central)
Store locally
(in local repository)
Use in build
(during compile/test/package)
Use in build
(artifact resolved and used)

---

15. Transitive Dependencies

Maven does more than download directly declared dependencies.

Suppose:

Your Application
↓
Library A
↓
Library B

If your application declares Library A, and Library A requires Library B, Maven can resolve Library B automatically.

This is known as a **transitive dependency**.

Tip

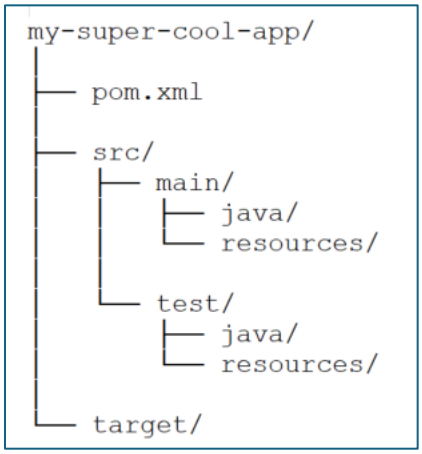
Dependency management is therefore not simply "downloading JAR files." Maven builds and resolves a **dependency graph**.

---

16. Maven Project Structure

Maven defines a standardized directory structure.

A typical project resembles:



For certain traditional web applications, you may also encounter: **`src/main/webapp/`**

16.1 pom.xml

Located in the project root:

```
my-super-cool-app/
└── pom.xml
```

This is Maven's primary project configuration file.

16.2 src/main/java

Application Java source code belongs here.

```
src/
└── main/
    └── java/
```

For example: **`src/main/java/com/example/myapp/`**

16.3 src/main/resources

Application resources and configuration files are normally stored here.

For a Spring Boot project, this commonly includes **`application.properties`**

16.4 src/test/java

Java test classes are stored here.

```
src/
└── test/
    └── java/
```

16.5 src/test/resources

Resources required specifically for testing can be placed here.

16.6 target

Maven places generated build output under:

`target/`

This may contain:

- compiled classes;
- generated files;
- packaged JAR/WAR files;
- test output.

Warning

`target/` contains generated build artifacts. Developers normally should not place source code there.

---

17. Why Maven's Standard Structure Matters

Standardization provides several advantages.

Easier Collaboration

A developer joining an unfamiliar Maven project can quickly identify:

`src/main/java` → **application source**

`src/main/resources` → **resources/configuration**

`src/test/java` → **tests**

`pom.xml` → **Maven project configuration**

`target` → **generated build output**

IDE Portability

Major Java IDEs understand Maven projects.

A project can therefore be moved between development environments without requiring developers to manually reconstruct its build path.

Increased Productivity

Developers spend less time configuring individual environments and more time working with the application.

---

18. Maven's Project Object Model — pom.xml

POM stands for:

Project Object Model

The file is named:

`pom.xml`

and is located at the root of the Maven project.

It acts as the project's Maven configuration blueprint.

A simplified POM might resemble:

```xml
<project>
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>mycoolapp</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <!-- project dependencies -->
    </dependencies>

    <build>
        <plugins>
            <!-- Maven plugins -->
        </plugins>
    </build>
</project>
```

Important areas include:

- project metadata;
- project coordinates;
- dependencies;
- build plugins.

---

19. Maven Coordinates

Maven identifies artifacts using coordinates.

Three particularly important values are:

GAV

where:

- **G** = groupId
- **A** = artifactId
- **V** = version

19.1 Group ID

The groupId generally identifies the organization or namespace responsible for the project.

For example:

<groupId>com.example</groupId>

Reverse-domain notation is commonly used.

19.2 Artifact ID

The artifactId identifies the particular project or module.

<artifactId>mycoolapp</artifactId>

19.3 Version

The version identifies a specific release:

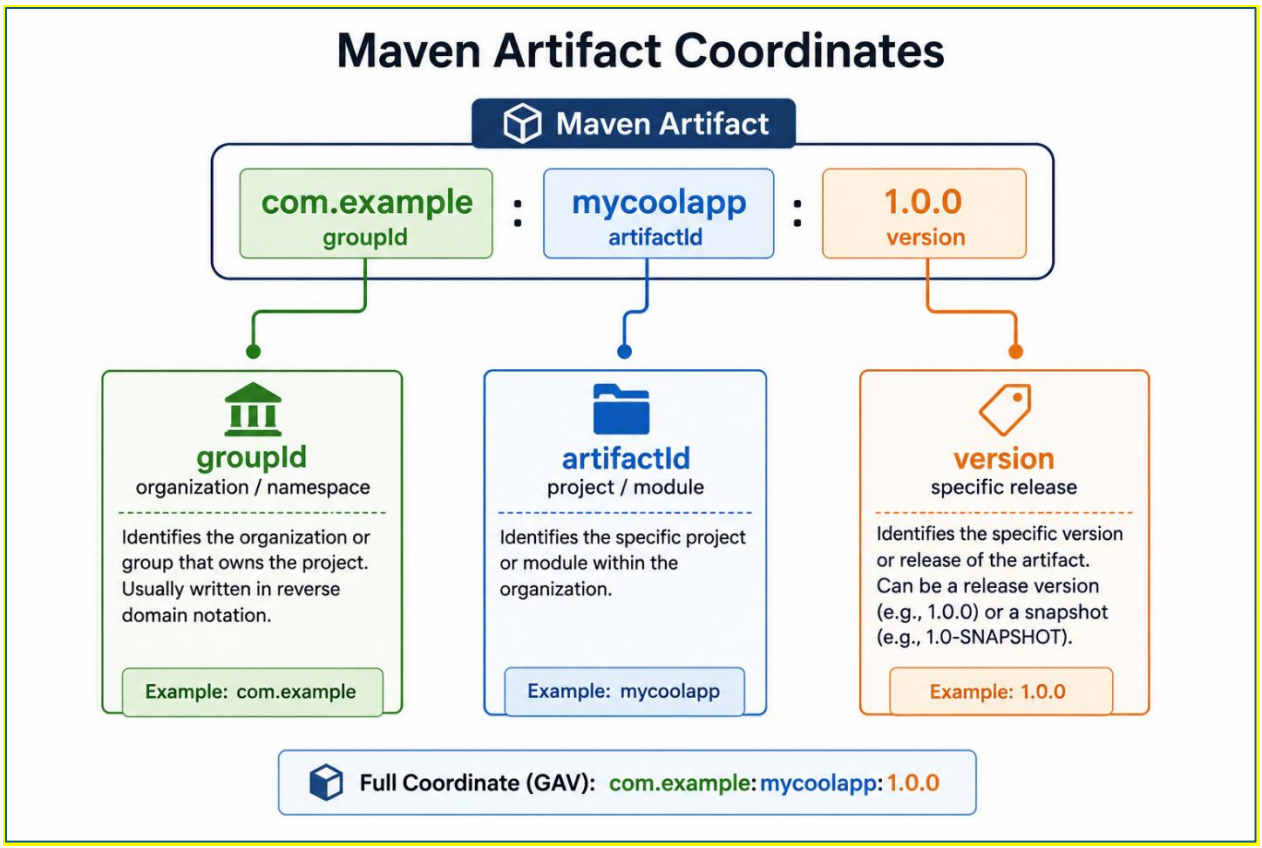
<version>1.0.0</version>

A project under development may use a snapshot version:

<version>1.0-SNAPSHOT</version>

Together:

```
com.example : mycoolapp : 1.0-SNAPSHOT
    ↑             ↑             ↑
 groupId      artifactId      version
```



---

20. Maven Dependencies

Dependencies are declared in the POM.

Conceptually:

```
<dependencies>

    <dependency>
        <groupId>...</groupId>
        <artifactId>...</artifactId>
        <version>...</version>
    </dependency>

</dependencies>
```

Maven uses these coordinates to locate the appropriate artifacts.

Dependencies can be obtained from repositories such as **Maven Central (**`mvnrepository.com`**).**

---

21. Spring Initializr

Now that we understand Maven, we can use **Spring Initializr** to generate a Spring Boot project.

Spring Initializr is available through the Spring website and generates a starter project based on developer-selected options.

It can generate projects using:

- Maven;
- Gradle.

For this course material, we focus on Maven.

---

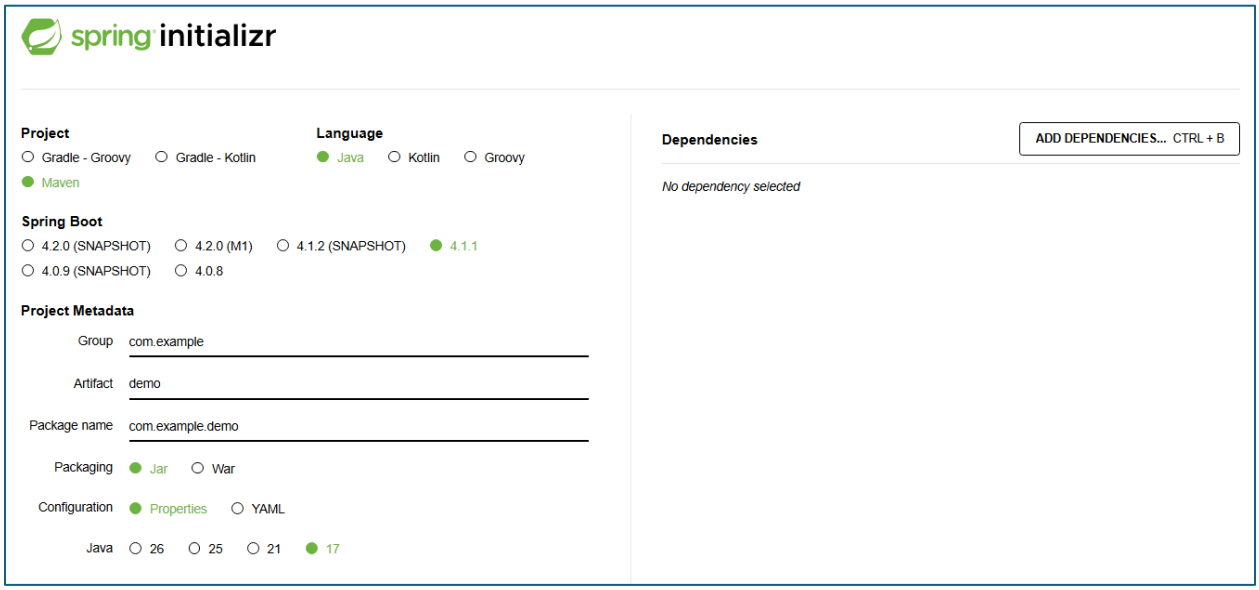
22. Lab: Generate a Spring Boot Project

Step 1 — Open Spring Initializr

Navigate to:

`start.spring.io`

You will see options for configuring the project.



Step 2 — Select Maven

Select:

```
Project: Maven
```

This tells Spring Initializr to generate a Maven project containing a pom.xml.

Step 3 — Select Java

Select:

```
Language: Java
```

Step 4 — Select a Spring Boot Version

Select the appropriate Spring Boot version offered by Spring Initializr.

Tip

In a classroom environment, use the version specified by the instructor so that all students work with a consistent environment.

Step 5 — Configure Project Metadata

Configure fields such as:

```
Group:    com.example
```

```
Artifact: mycoolapp
```

These values contribute to the project's Maven coordinates and package structure.

Step 6 — Add Dependencies

Add the dependencies required by the application.

For the REST example later in these notes, the project needs appropriate Spring web support.

The generated POM will contain the corresponding Spring Boot dependency configuration.

Step 7 — Generate the Project

Select **Generate**.

Spring Initializr creates a ZIP archive containing the starter project.

For example:

```
mycoolapp.zip
```

Step 8 — Extract the Project

Unzip the downloaded file into your development directory.

For example:

```
projects/
└── mycoolapp/
```

Step 9 — Import the Maven Project

Open or import the extracted project into your IDE.

Because the project contains:

```
pom.xml
```

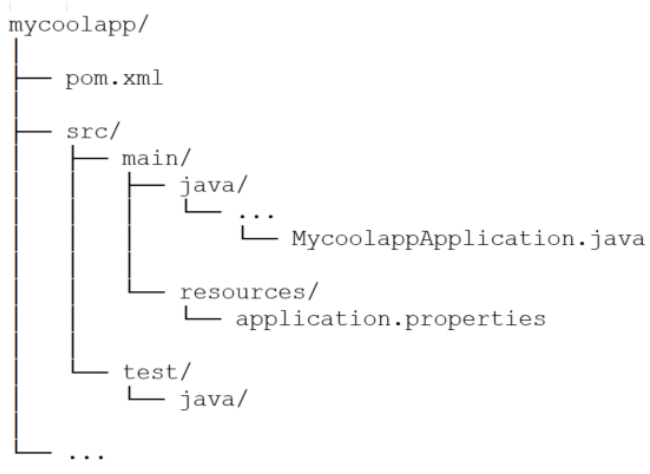
the IDE can recognize it as a Maven project.

Maven will then resolve the required dependencies.

---

23. Examine the Generated Spring Boot Project

A typical generated project contains a structure similar to:



Notice how this structure corresponds directly to the Maven conventions discussed earlier.

---

24. Running the Spring Boot Application

Spring Initializr generates a main application class.

When the application starts, Spring Boot initializes the Spring application and, for a web application, starts its embedded server.

A typical console message will indicate that the server has started.

The default HTTP port commonly used is:

`8080`

The application can then be accessed at:

`http://localhost:8080`

At this stage, however, the application may not yet have a handler for the root URL.

That is what we will implement next.

---

25. Lab: Create Your First REST Controller

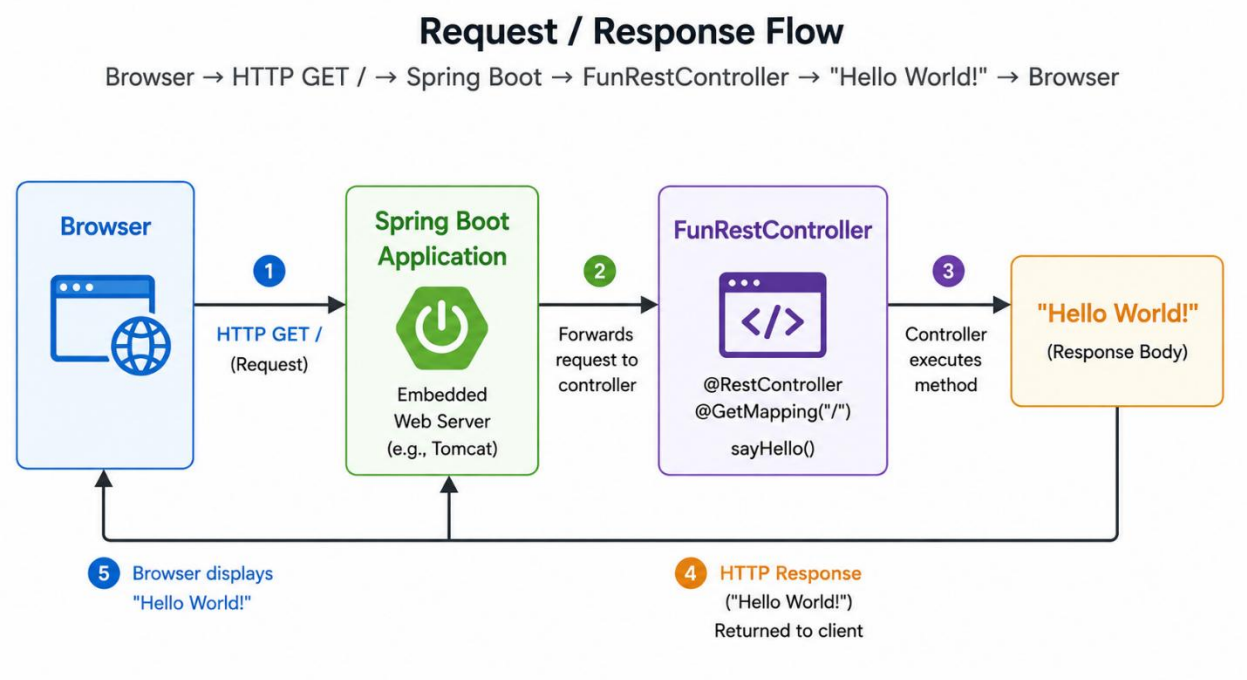
Our goal is to create an endpoint:

`GET /`

that returns:

`Hello World!`

The request flow will be:



---

26. Step 1 — Create a REST Package

Inside the application's Java package structure, create a package named:

`rest`

For example:

`com.example.mycoolapp.rest`

Important

Keep the controller package underneath the application's base package so that Spring Boot's default component scanning can discover the controller.

---

27. Step 2 — Create the Controller Class

Inside the rest package, create:

`FunRestController.java`

Initially:

```
public class FunRestController {

}
```

---

28. Step 3 — Add @RestController

Add:

`@RestController`

above the class declaration:

```
@RestController
public class FunRestController {


}
```

Import:

```
import org.springframework.web.bind.annotation.RestController;
```

The complete code so far becomes:

```
package com.example.mycoolapp.rest;

import org.springframework.web.bind.annotation.RestController;

@RestController
public class FunRestController {

}
```

What Does @RestController Do?

@RestController identifies this class as a Spring web component whose handler methods can return data directly in HTTP responses.

It also allows Spring's component-scanning mechanism to discover and register the controller as a Spring-managed bean.

---

29. Step 4 — Create the GET Endpoint

We now want this URL:

`/`

to invoke a Java method.

Add:

```
@GetMapping("/")
```

and create the method:

```
public String sayHello() {
    return "Hello World!";
}
```

The complete controller becomes:

```
package com.example.mycoolapp.rest;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class FunRestController {

    @GetMapping("/")
    public String sayHello() {
        return "Hello World!";
    }
}
```

---

30. Understanding the Controller

Consider:

```
@RestController
public class FunRestController {
```

@RestController tells Spring that this class contains REST endpoint handlers.

Next:

```
@GetMapping("/")
```

maps an HTTP **GET** request for:

```
/
```

to the method immediately below it.

Finally:

```
public String sayHello() {
    return "Hello World!";
}
```

returns the response body:

```
Hello World!
```

The complete mapping is therefore:

```
GET /
 ↓
sayHello()
 ↓
"Hello World!"
```

Pitfall

@RestController does not by itself define a URL. A request-mapping annotation such as @GetMapping is required to associate a handler method with an HTTP request path.

---

31. Step 5 — Run the Application

Start the Spring Boot application from the IDE.

Watch the console and confirm that the application starts successfully.

For the default configuration, the embedded web server typically listens on:

```
8080
```

---

32. Step 6 — Test the Endpoint

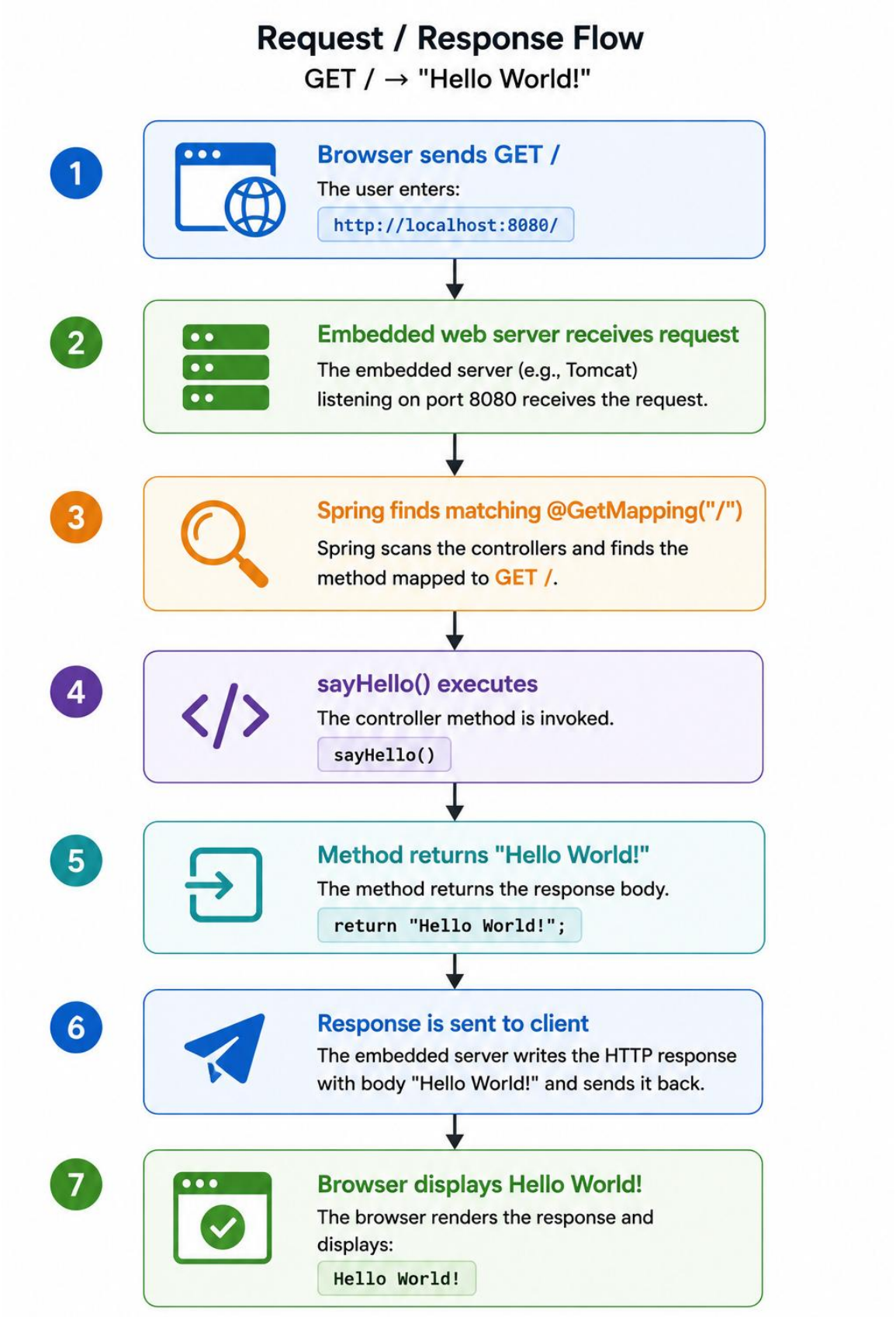
Open a browser and navigate to:

```
http://localhost:8080/
```

The browser should display:

```
Hello World!
```

The complete interaction is:


Request / Response Flow
GET / → "Hello World!"
1
Browser sends GET /
The user enters:
http://localhost:8080/
2
Embedded web server receives request
The embedded server (e.g., Tomcat) listening on port 8080 receives the request.
3
Spring finds matching @GetMapping("/")
Spring scans the controllers and finds the method mapped to GET /.
4
sayHello() executes
The controller method is invoked.
sayHello()
5
Method returns "Hello World!"
The method returns the response body.
return "Hello World!";
6
Response is sent to client
The embedded server writes the HTTP response with body "Hello World!" and sends it back.
7
Browser displays Hello World!
The browser renders the response and displays:
Hello World!

---

33. Troubleshooting the Lab

Problem: Port 8080 Is Already in Use

Another application may already be listening on port `8080`.

Warning

Only one process can normally listen on the same TCP port/interface combination at a time. If Spring Boot reports that port 8080 is already in use, stop the conflicting process or configure the application to use another port.

Problem: The Root URL Does Not Return Hello World!

Check:

```
@GetMapping("/")
```

and verify that the controller is inside a package discoverable by Spring's component scan.

Problem: @RestController or @GetMapping Cannot Be Resolved

Verify that:

- the required Spring web dependency was selected;
- Maven successfully resolved the dependencies;
- the appropriate imports exist.

The expected imports are:

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

---

34. Putting Everything Together

At this point, several technologies that initially appear separate form one development workflow:

Spring Framework
↓

Provides the underlying application framework

↓

Spring Boot

↓

Simplifies configuration and application startup

↓

Spring Initializr

↓

Generates the starter project

↓

Maven

↓

Manages dependencies and the build

↓

Spring Boot Application

↓

Embedded Web Server

↓

@RestController

↓

@GetMapping("/")

↓

HTTP Response

This relationship is fundamental to understanding modern Spring development.